Multi-Cluster Fluctuating Two-Ray Fading Model approximation with Conditional GANs

1st José Ocampo
Master's Student in Artificial Intelligence
Universidad San Francisco de Quito
Quito, Ecuador
docampor@estud.usfq.edu.ec

***Abstract*—The Multi-Cluster Fluctuating Two-Ray (MFTR) model provides a flexible description of wireless fading but yields signal-envelope distributions whose shape can change sharply with its parameters. We study whether a conditional generative adversarial network (cGAN) can learn these distributions and generate samples for parameters inside and outside training range.**

We train a lightweight MLP-based cGAN on synthetic MFTR envelope samples, using a continuous condition input (here, sweeping the fading parameter μ while keeping the remaining parameters fixed). We evaluate conditional sample quality against the theoretical MFTR reference using QQ plots and distributional metrics: Kolmogorov–Smirnov statistic, 1D Wasserstein distance, and quantile MSE/MAE.

Results show that within the training range ($\mu \in [1, 9]$), the generated conditional samples closely match the theoretical distribution. Extrapolation beyond the training range remains feasible but is less reliable, with similar median errors yet increased worst-case discrepancies at larger μ. Training curves (losses and discriminator probabilities) stabilize early and remain nearly constant, indicating that these diagnostics alone are not sufficient to assess final conditional fit quality.

***Index Terms*—GANs, wireless communications, fading models, MFTR**

I. Introduction

The estimation of an underlying probability distribution from data is a fundamental problem in many fields, including wireless communications. Since a population can be fully characterized by its probability distribution, being able to estimate this distribution from data is crucial for understanding and modeling complex phenomena, generate new samples, and make predictions under given conditions, say for instance environmental factors related to the propagation of radio signals across a medium.

On way to do so is to use Generative Adversarial Networks (GANs), a family of generative models that generate new samples from a target distribution by implicitly learning their underlying distribution [3]. They consist of two neural networks, a generator and a discriminator, that are trained in an adversarial manner. The generator takes random noise as input and produces synthetic samples, while the discriminator tries to distinguish between real and fake samples. Through this process, the generator learns to produce samples that are indistinguishable from real data. GANs have been successfully applied in various domains, including image generation [4], time series simulation [2] and audio synthesis [1].

In this work, we focus our attention in a variant of GANs called conditional GANs (cGANs), which allow for the generation of samples conditioned on specific input variables, enabling more control over the generated data [5]. Although the original cGANs paper uses categorical labels as the condition input, there have been several works that have successfully used continuous variables as conditions, such as [2] and [7]. Our goal is to use them to approximate the distribution of a complex fading model, the Multi-Cluster Fluctuating Two-Ray (MFTR) fading model [6].

The MFTR model is a generalization of the Fluctuating Two-Ray (FTR) fading model and the $\kappa - \mu$ fading model, all of which attempt to model the amount of power received at a given location when a radio signal travels and interacts with the environment in several ways. It depends on 5 parameters, $\Delta, m, \mu, K, \bar{\gamma}$ (see Fig. 1) that allow extensive control of the shape of the distribution, including a transition from an unimodal to a bimodal distribution, which makes it a good candidate for testing the ability of cGANs to learn complex distributions.

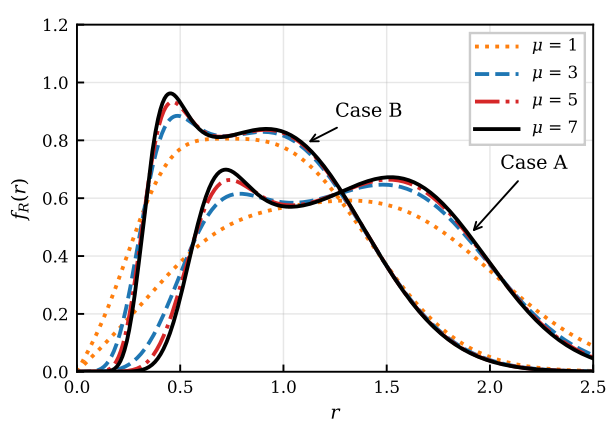

Fig. 1. PDF of the MFTR signal envelope by varying μ in two different scenarios: Case A: $\Delta = 0.9$, $m = 8$, $K = 8$, $\bar{\gamma} = 2$ and Case B: $\Delta = 0.9$, $m = 4$, $K = 15$, $\bar{\gamma} = 1$. Reproduced from [6].

This work is a first step towards the ultimate goal of learning an empirical distribution from real data taken from experiments on wireless channels, which would allow us to generate new samples with different parameters. This requires

our GAN to be able to interpolate and extrapolate, which we explore in the following sections.

II. Methodology

We train a conditional GAN to generate MFTR envelope samples conditioned on the MFTR parameters. Each training example is a pair $(x, \mathbf{c})$, where $x \in \mathbb{R}$ is a single amplitude sample and $\mathbf{c} = [m, \mu, K, \Delta, \bar{\gamma}]$ is the conditioning vector.

A. Architecture

The model has two multilayer perceptrons (MLPs): a generator G and a discriminator D. The generator takes a noise vector $\mathbf{z} \in \mathbb{R}^{1024}$ and the condition $\mathbf{c}$, embeds $\mathbf{c}$ with a small MLP (Linear–ReLU–Linear) to a 32-dimensional vector, concatenates $[\mathbf{z}, \mathrm{emb}(\mathbf{c})]$, and passes it through an MLP with hidden sizes 512–256–128 to output one value. All intermediate layers have LeakyReLU with $\beta 0.2$ as activation function, and `Softplus` activation is used at the output to keep generated amplitudes positive. The discriminator embeds the condition the same way, concatenates $[x, \mathrm{emb}(\mathbf{c})]$, and uses an MLP with hidden sizes 512–256–128 and the same activation functions as the generator (except the softplus) to output a single logit.

B. Loss function and training tricks

We use the standard GAN objective implemented with `BCEWithLogitsLoss` (numerically stable because D outputs logits). For the discriminator we use

$$\mathcal{L}_D = \mathrm{BCE}(D(x, \mathbf{c}), 0.9) + \mathrm{BCE}(D(G(\mathbf{z}, \mathbf{c}), \mathbf{c}), 0), \quad (1)$$

where the target for real samples is set to 0.9 (one-sided label smoothing). For the generator we use the non-saturating loss

$$\mathcal{L}_G = \mathrm{BCE}(D(G(\mathbf{z}, \mathbf{c}), \mathbf{c}), 1). \quad (2)$$

We update D every step and update G every k steps (here $k = 2$). Linear layers are initialized with Xavier uniform initialization.

C. Training setup

Training is done for 150 epochs using Adam for both networks, with $\beta_1 = 0.5$ and $\beta_2 = 0.999$. We use learning rates $\mathrm{lr}_G = 2.8 \times 10^{-4}$ and $\mathrm{lr}_D = 1.12 \times 10^{-3}$ and a batch size of 512.

D. Condition normalization and embedding

The five condition parameters are min–max normalized to $[-1, 1]$ using the fixed training ranges, independently per dimension. If a parameter is fixed (low=high), its normalized value is set to 0. Both G and D use the same conditioning mechanism: the normalized condition vector is projected to a learned embedding of dimension 32 using the small embedding MLP described above.

E. Dataset generation (uniform combos and varying only μ)

We construct a conditional dataset by sampling a set of parameter combinations (*combos*) uniformly within the chosen ranges, and then generating many MFTR samples for each combo. In this report we focus on a simplified experiment where only μ varies uniformly in $[1, 9]$, while the other parameters are fixed ($m = 8$, $K = 9$, $\Delta = 0.9$, $\bar{\gamma} = 1$). This setting is useful because changing μ can move the MFTR envelope PDF from unimodal to bimodal, so it tests whether the model can smoothly adapt its output shape. We sample 128 training combos and 32 validation combos uniformly (disjoint split). For each combo we draw 10,000 training samples (and 1,000 validation samples) from the analytic MFTR sampler (amplitude mode). The dataset stores the combos in a tensor of shape (combos, 5) and returns samples with their corresponding condition by repeating each combo for its associated samples.

III. Results

Our evaluation compares generated conditional samples against theoretical MFTR samples using (i) interpolation cases (conditions inside the training range) and (ii) extrapolation cases (conditions outside the training range). For each case, we use $n = 5000$ samples and compute the Kolmogorov–Smirnov statistic (KS), 1D Wasserstein distance, and the MSE/MAE between sorted empirical quantiles.

A. Conditional interpolation vs extrapolation

Fig. 2 and Fig. 3 show three representative examples for interpolation and extrapolation, respectively. Visually, interpolation is consistently better: the QQ points stay closer to the diagonal and the histograms/KDEs follow the theoretical MFTR shape more closely across different condition values. The extrapolation really starts to suffer when μ goes much farther than 9, the training upper limit, since the distribution then becomes unimodal again with a very long tail, which was unseen during training. We also observe that for μ very close to 1, the model still tries to produce a bimodal distribution, which is a mismatch with the true unimodal MFTR shape in that case. This suggests that the model has not fully learned the underlying data generation process, but rather has learned to interpolate between the training shapes without fully understanding how the shape changes with μ.

This is also reflected in the metrics in Table I.The median errors are similar in- vs out-of-range, but the worst-case errors increase out-of-range (e.g., max Wasserstein ≈ 0.109 vs ≈ 0.050).

TABLE I
Conditional metrics (min / median / max) for interpolation (in-range) and extrapolation (out-of-range).

Metric	In-range (interp.)	Out-of-range (extra.)
KS stat.	0.021 / 0.064 / 0.075	0.055 / 0.065 / 0.146
Wasserstein	0.016 / 0.045 / 0.050	0.025 / 0.045 / 0.109
MSE	3.87×10^{-4} / 0.0024 / 0.0031	0.0012 / 0.0026 / 0.014
MAE	0.016 / 0.045 / 0.050	0.025 / 0.045 / 0.109

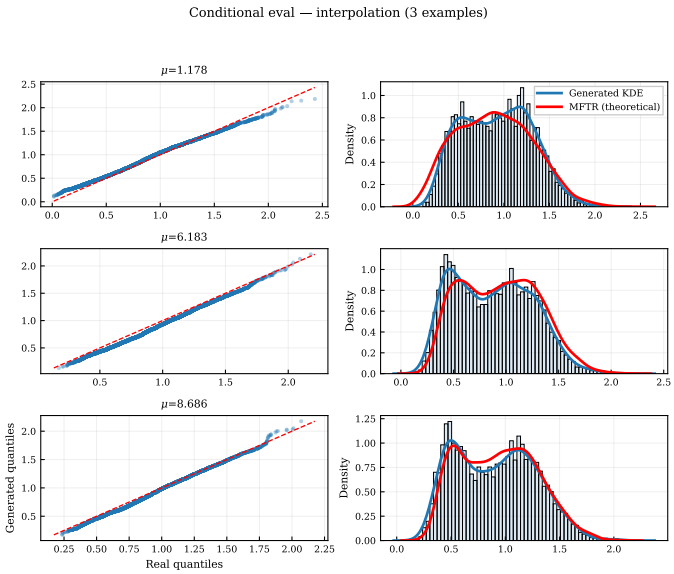

Fig. 2. Conditional evaluation (interpolation): three in-range examples.

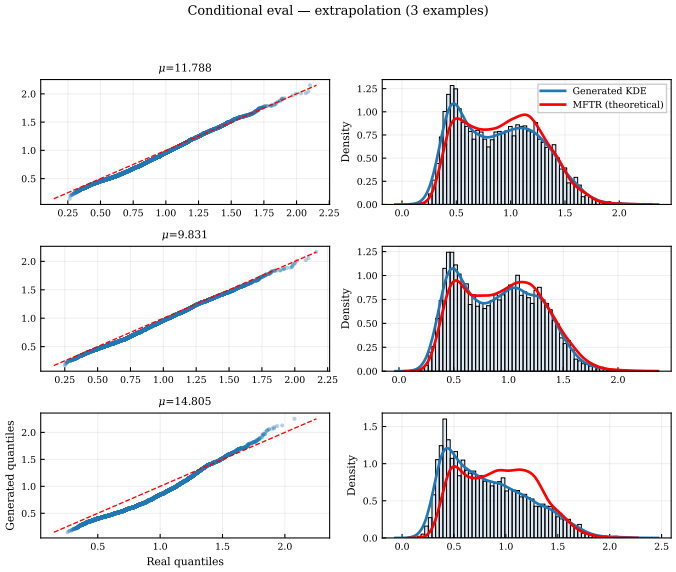

Fig. 3. Conditional evaluation (extrapolation): three out-of-range examples.

B. Training dynamics (losses and discriminator probabilities)

The discriminator and generator losses stabilize very early (Fig. 4). Across the 150 epochs, the discriminator loss stays close to 1.376 (min 1.373, max 1.377) and the generator loss stays close to 0.80 (min 0.796, max 0.834). Based on these curves, both losses reach a clear plateau by approximately epoch 11. Even though the losses do not show an obvious downward trend after that point, training for fewer epochs produced noticeably worse conditional fits (verified experimentally). This suggests that in this setting the losses are not a reliable indicator of the final sample quality.

The predicted discriminator probabilities for real and fake samples also remain stable and close to each other (Fig. 5).

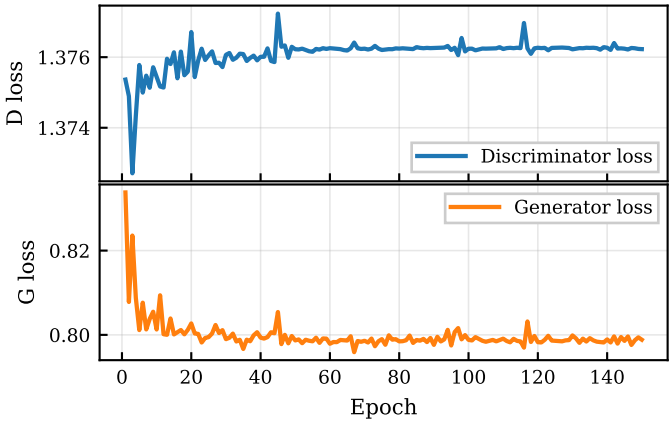

Fig. 4. Training curves for generator and discriminator losses.

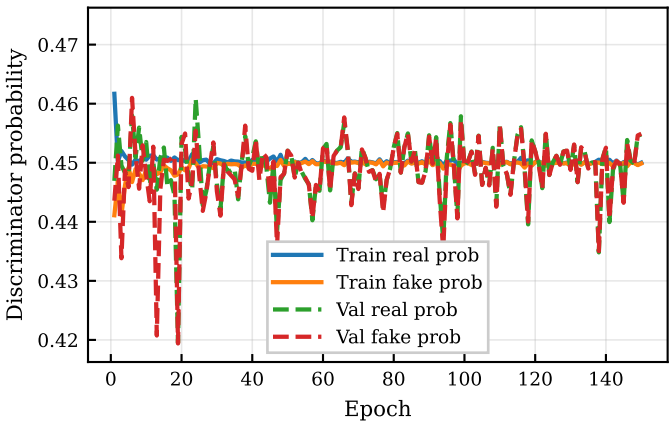

Fig. 5. Discriminator predicted probabilities for real and fake samples.

On the training set, both probabilities oscillate around 0.45 during most training (means: 0.450 for real and 0.450 for fake). On the validation set, the probabilities are also close to 0.45 on average (means: 0.449 for both), with slightly larger oscillations.

IV. Conclusion

We showed that a conditional GAN can learn a practical approximation of the MFTR fading envelope distribution when conditioned on one of its parameters. Within the training range ($\mu \in [1, 9]$), the generated conditional samples closely match the theoretical MFTR reference in both the QQ plots and distribution shape.

Generalization beyond the training range is more fragile: while median errors can remain comparable, the worst-case discrepancies increase for extrapolation (notably at larger μ values beyond the training upper limit). Finally, training curves (losses and discriminator probabilities) stabilize early and remain nearly constant, indicating that these diagnostics alone are not sufficient to judge the final conditional fit quality.

V. Future work

There are many things in the current work that could be improved or extended in the future. Here are some ideas:

- Use the Wasserstein distance as a monitoring metric instead of loss values, which are not directly comparable between models and can be unstable. This would potentially allow for checkpointing and early stopping.
- Explore other GAN loss functions, particularly the MMD GAN as it seems to produce better results [7].
- Add some type of normalization to the generator and discriminator outputs, such as batch normalization or layer normalization, which could help stabilize training and improve performance. Since we are dealing with a 1D distribution with a single feature, batch normalization seems most appropriate.
- Experiment with the hyperparameters of the model, such as the learning rates, batch size, and architecture (number of layers and hidden units). A more systematic hyperparameter search could lead to better results, specially since the losses seem to be plateauing too early.
- Repeat the experimet with real, experimental data instead of synthetic MFTR samples. This would be a more realistic test of the model's ability to capture the underlying distribution and generalize to new conditions.
- Extend the model to handle multiple varying parameters at the same time, instead of just μ. This would allow us to test the model's ability to learn more complex relationships between the parameters and the output distribution.
- Explore other types of generative models, such as variational autoencoders (VAEs) or normalizing flows, which might be better suited for modeling complex distributions and could potentially provide better performance than GANs in this setting.

References

[1] Chris Donahue, Julian McAuley, and Miller Puckette. *Adversarial Audio Synthesis*. 2019. arXiv: 1802.04208 [cs.SD]. URL: https://arxiv.org/abs/1802.04208.

[2] Rao Fu et al. "Time series simulation by conditional generative adversarial net". In: *arXiv preprint arXiv:1904.11419* (2019).

[3] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: https://arxiv.org/abs/1406.2661.

[4] Tero Karras, Samuli Laine, and Timo Aila. *A Style-Based Generator Architecture for Generative Adversarial Networks*. 2019. arXiv: 1812.04948 [cs.NE]. URL: https://arxiv.org/abs/1812.04948.

[5] Mehdi Mirza and Simon Osindero. "Conditional generative adversarial nets". In: *arXiv preprint arXiv:1411.1784* (2014).

[6] José David Vega Sánchez et al. *The Multi-cluster Fluctuating Two-Ray Fading Model*. 2023. arXiv: 2212.02448 [cs.IT]. URL: https://arxiv.org/abs/2212.02448.

[7] Manzil Zaheer et al. "GAN connoisseur: Can GANs learn simple 1D parametric distributions". In: *Proceedings of the 31st Conference on Neural Information Processing Systems*. 2017, pp. 1–6.

[8] Yufeng Zheng, Yunkai Zhang, and Zeyu Zheng. *Continuous Conditional Generative Adversarial Networks (cGAN) with Generator Regularization*. 2021. arXiv: 2103.14884 [cs.LG]. URL: https://arxiv.org/abs/2103.14884.